

Software Defined Networking - Design Principles

Richard T. B. Ma

School of Computing

National University of Singapore

Material from: Scott Shenker (UC Berkeley), Nick McKeown (Stanford), Larry Peterson (Open Networking Foundation)

CS4226: Internet Architecture

What is Software Defined Networking?

- ❑ A new approach to do networking
 - ❖ to implement and manage networks
 - ❖ based on new fundamental principles
 - ❖ impacts the pace of innovation
- ❑ How does it contrast with traditional networking?
- ❑ Before knowing what it is, let us understand why we need it in the first place
 - ❖ what is wrong with the current Internet?

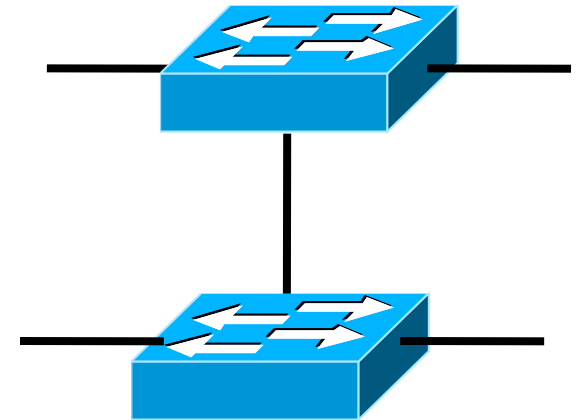
The Internet: A Remarkable Story

- ❑ Tremendous success
 - ❖ from research experiment to global infrastructure
- ❑ Brilliance of under-specifying
 - ❖ network: best-effort packet delivery
 - ❖ hosts: arbitrary applications
- ❑ Enables innovation in applications
 - ❖ web, P2P, VoIP, social networks, virtual worlds
- ❑ But, change is easy only at the edge... ☹



Inside the Net: A Different Story

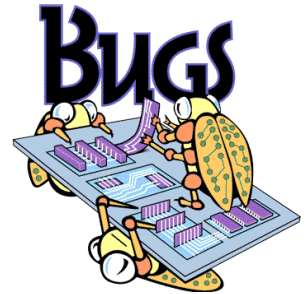
- ❑ Closed equipment
 - ❖ software bundled with hardware
 - ❖ vendor-specific interfaces
- ❑ Over specified
 - ❖ slow protocol standardization
- ❑ Few people can innovate
 - ❖ equipment vendors write the code
 - ❖ long delays to introduce new features



Impacts performance, security, reliability, cost...

Networks are Hard to Manage

- ❑ Operating a network is expensive
 - ❖ more than half the cost of a network
 - ❖ yet, operator error causes most outages
- ❑ Buggy software in the equipment
 - ❖ routers with 20+ million lines of code
 - ❖ cascading failures, vulnerabilities, etc.
- ❑ The network is “in the way”
 - ❖ especially a problem in data centers
 - ❖ ... and home networks



Networks Vs. Other Systems

□ Networks are hard to manage

- ❖ computation and storage have been virtualized
 - creating more flexible and manageable infrastructure
- ❖ networks are still notoriously hard to manage
 - still heavily rely on network administrators

□ Networks are hard to evolve

- ❖ ongoing innovation in systems software
 - new programming languages, operating systems, etc.
- ❖ networks are stuck in the past
 - routing algorithms change very slowly
 - network management extremely primitive

Why Does Networking Lag Behind?

- ❑ Networks used to be simple: Ethernet, IP, TCP....
- ❑ New **control** requirements led to great complexity
 - ❖ isolation → VLANs, ACLs
 - ❖ traffic engineering → MPLS
 - ❖ packet processing → Firewalls, NATs
 - ❖ payload analysis → Deep packet inspection
- ❑ Ability to master complexity → a curse
 - ❖ extract simplicity is needed to build a discipline

Networking as a Discipline

- ❑ Other fields in “systems”: OS, DB, DS, etc.
 - ❖ Teach basic principles
 - ❖ Are easily managed
 - ❖ Continue to evolve

- ❑ Networking:
 - ❖ Teach big bag of protocols
 - ❖ Notoriously difficult to manage
 - ❖ Evolves very slowly

A failure from an academic point of view

A Good Example: Programming

- ❑ Machine languages: no abstractions
 - ❖ mastering complexity was crucial
- ❑ Higher-level abstractions in OS
 - ❖ memory (paging/segmentation), file system (namespace/directory), CPU (process/thread), network (socket) ...
- ❑ Modern languages: even more abstractions
 - ❖ data (object), control flow (first-class functions), memory (garbage collection), ...

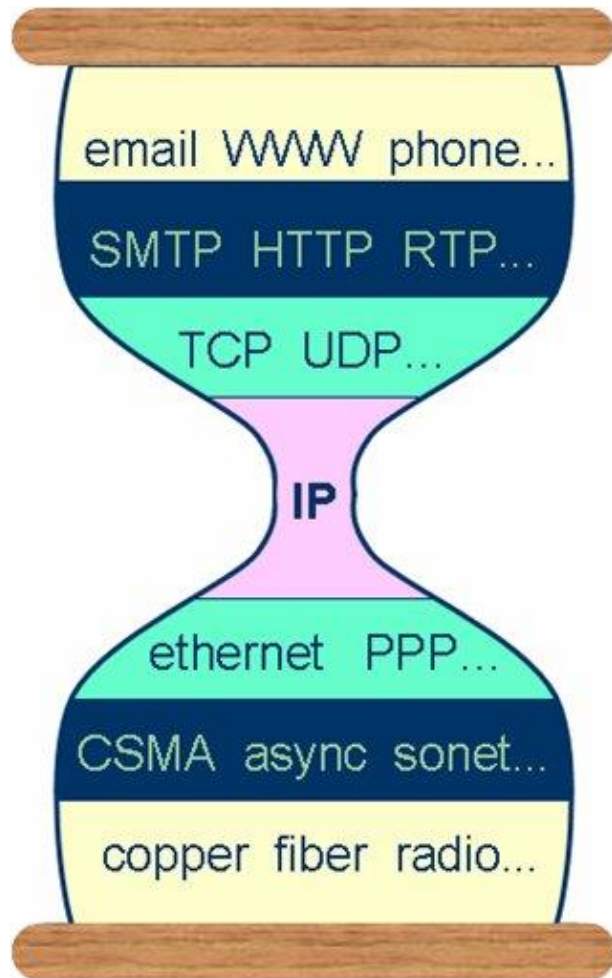
Abstraction is the key to extracting simplicity

“Modularity based on abstraction
is the way things are done!”

- Barbara Liskov (MIT)
- Turing Award Lecture 2009



Key to the Internet's Success



- ❑ Hourglass IP model
- ❑ Layered service abstractions (why is this important?)
 - decompose delivery into fundamental components
 - independent, compatible innovation at each layer
- ❑ Only for network edges

The Network Control Problem

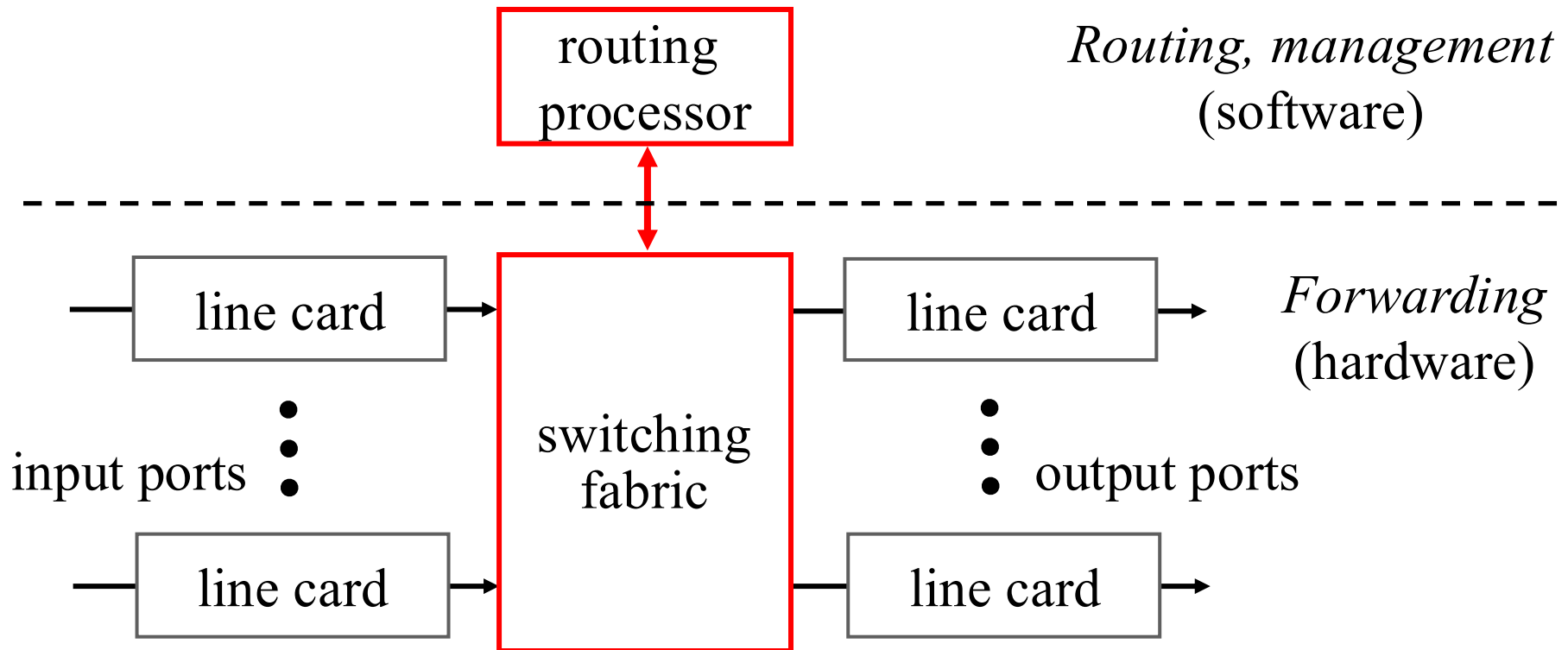
- ❑ Compute the configuration of each physical devices, e.g., forwarding tables
- ❑ Operate without communication guarantees
- ❑ Operate within given network-level protocol, e.g., RIP, OSPF

Only people who love complexity would find this a reasonable request!

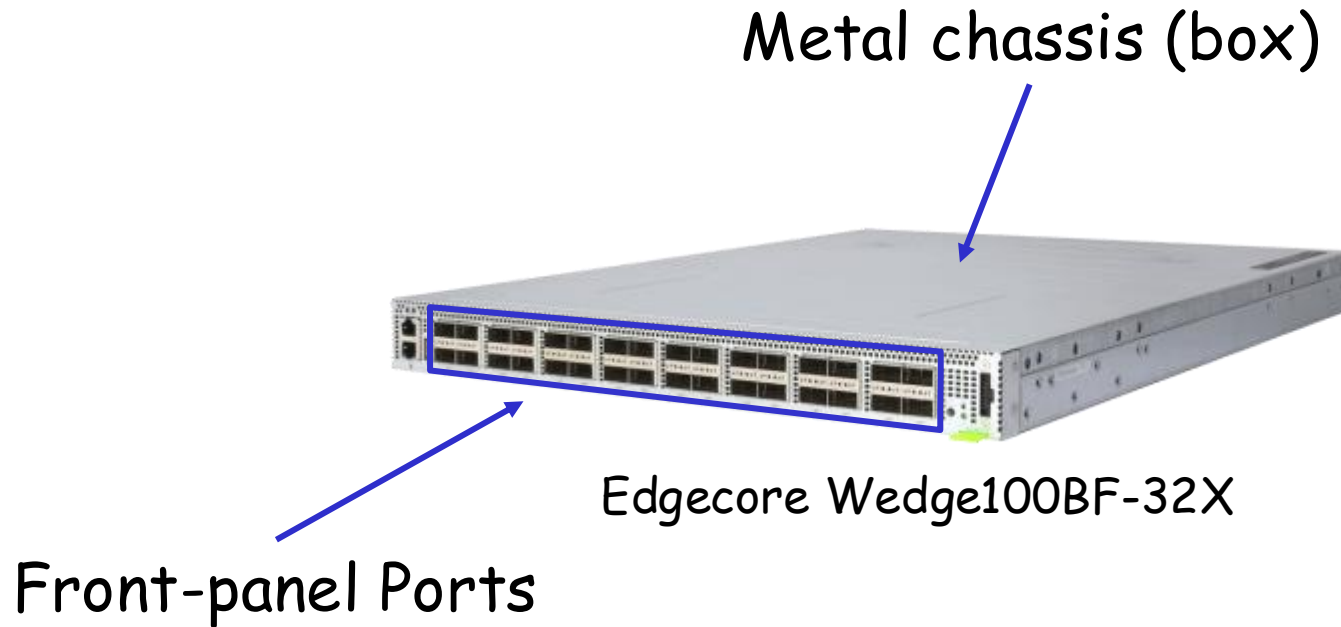
Router/Switch at the Core

two key functions:

- ❑ run *routing* algorithms/protocol (RIP, OSPF, BGP)
- ❑ *forwarding* datagrams from incoming to outgoing link



Network Switch



What's inside a network switch?

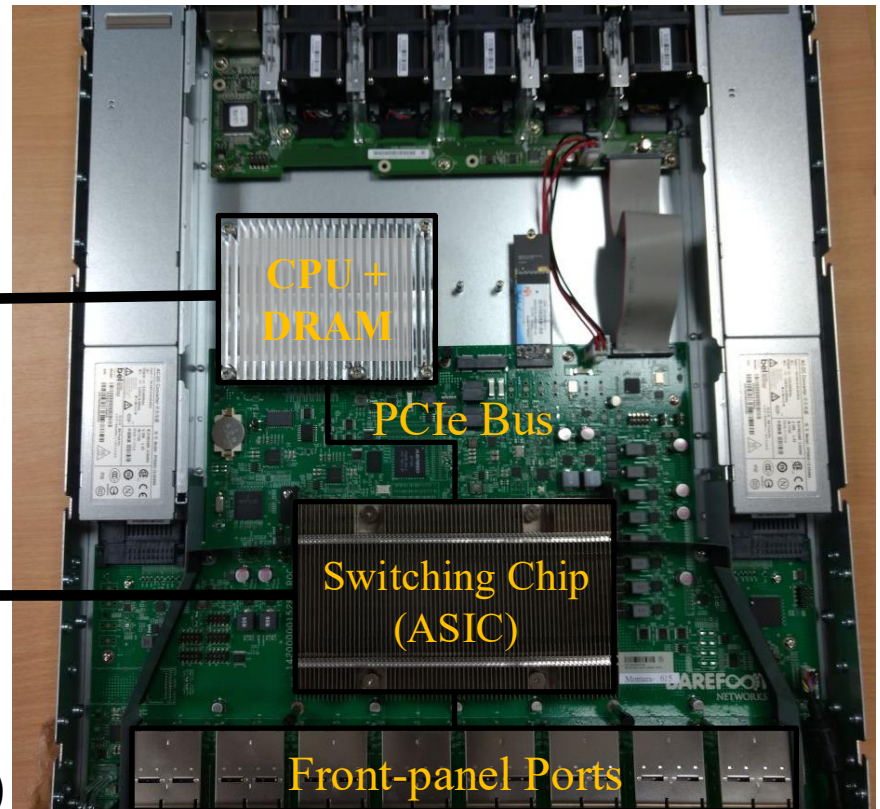
Control Plane

- Apps/Protocols: L2, BGP, etc.
- Operating System (*nix)
- ASIC driver



Data Plane

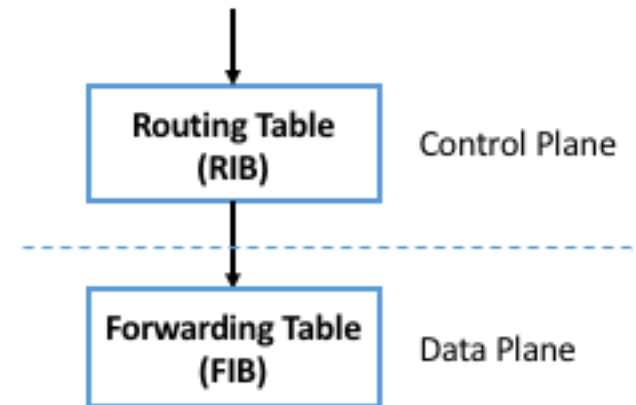
- Match-action tables
- Processes *almost* all traffic in/out of front-panel ports (performs "forwarding" function)



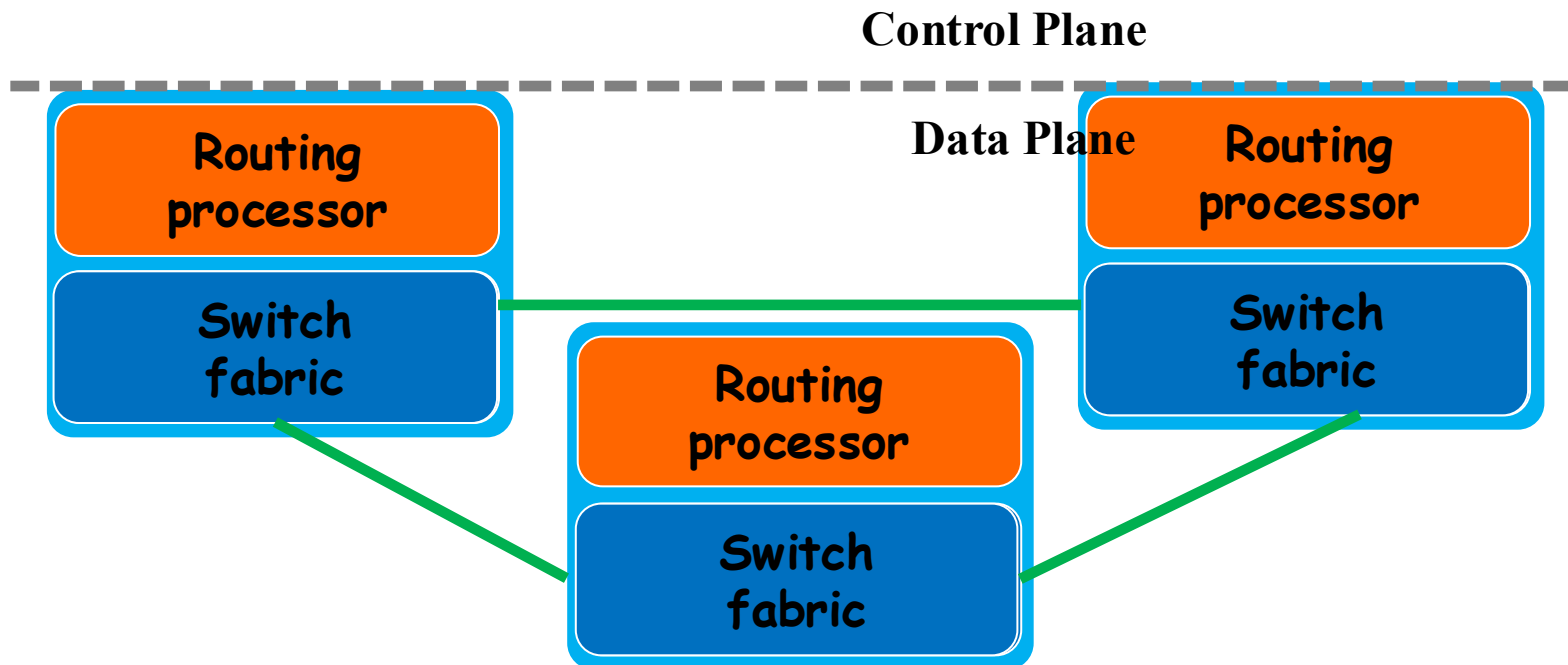
Edgecore Wedge100BF-32X

Control Plane vs Data Plane

- ❑ **Control Plane:** establish router state
 - ❖ determine how/where packets are forwarded
 - ❖ routing, traffic engineering, firewall
 - ❖ slow time-scales (per control event)
- ❑ **Data Plane:** process/deliver packets
 - ❖ based on state in routers and endpoints
 - ❖ e.g., IP, TCP, Ethernet, etc.
 - ❖ fast timescales (per-packet)

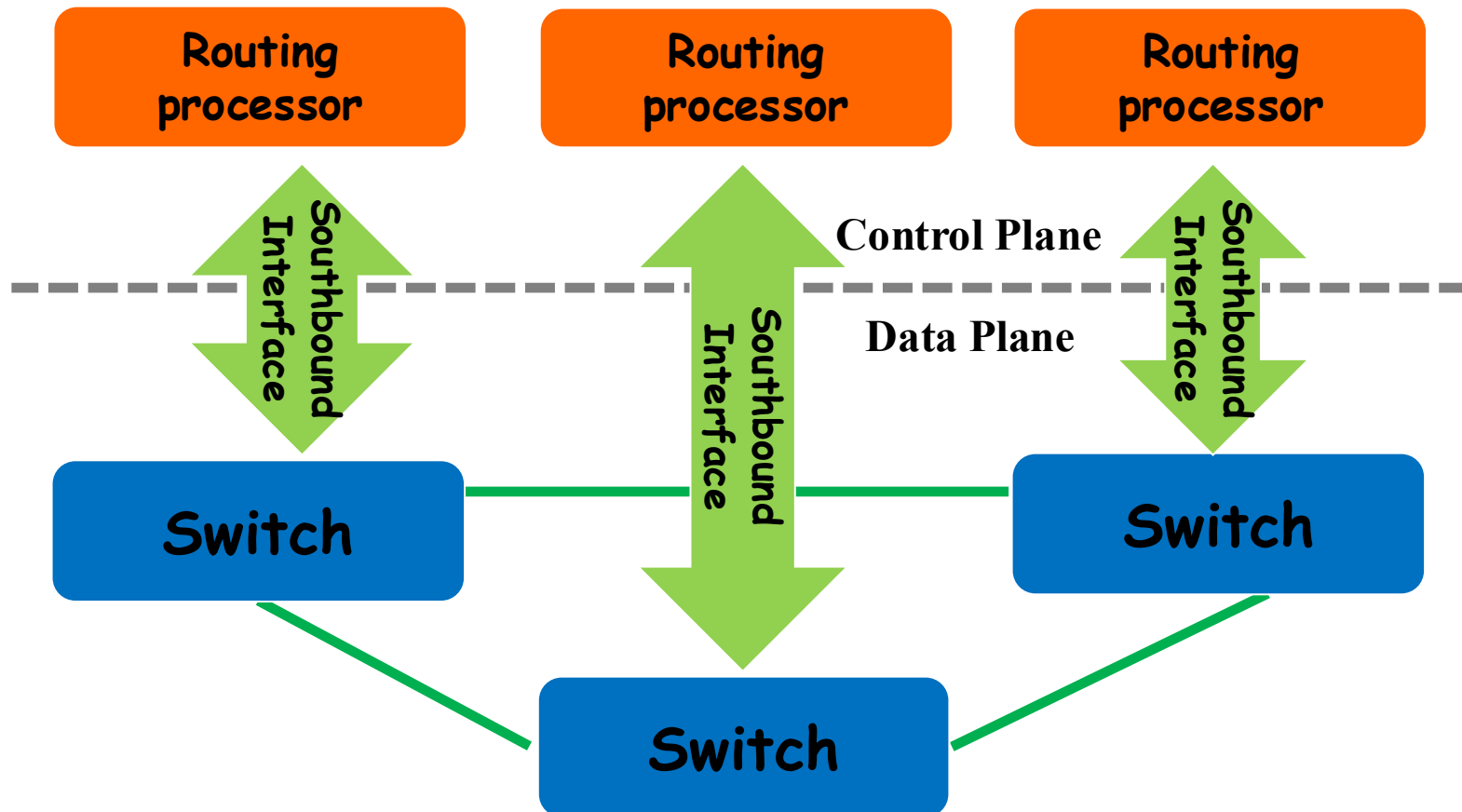


Separation of Control/Data Plane



Principle 1: Disaggregation

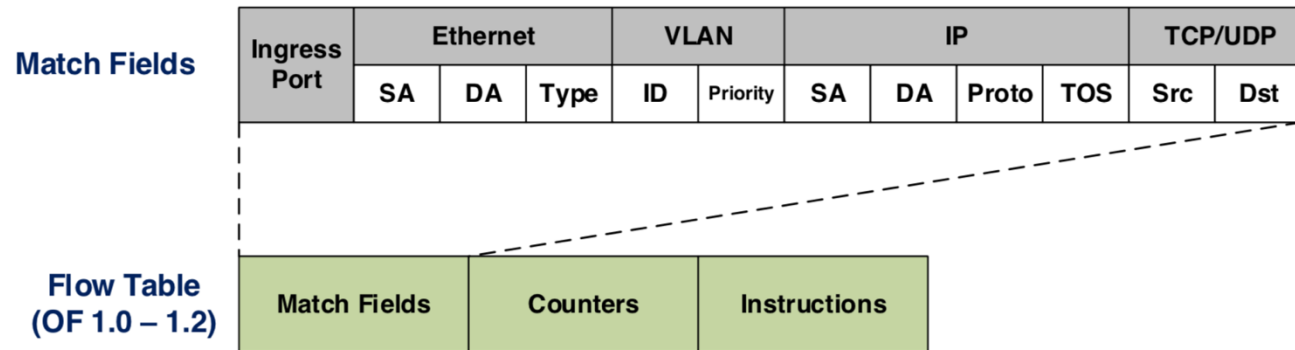
- ❑ should be codified in an open interface



Disaggregation - Implications

- ❑ Network operators are able to purchase control & data planes from vendors X & Y.
- ❑ Data plane consists of cheaper commodity forwarding devices (bare-metal switches)
- ❑ Needs to define a *forwarding abstraction*
 - ❖ general-purpose way for control plane to tell data plane to forward pkts in a particular way
 - ❖ e.g., OpenFlow's *Flow Rules* as a simple-but-powerful way to specify forwarding behavior

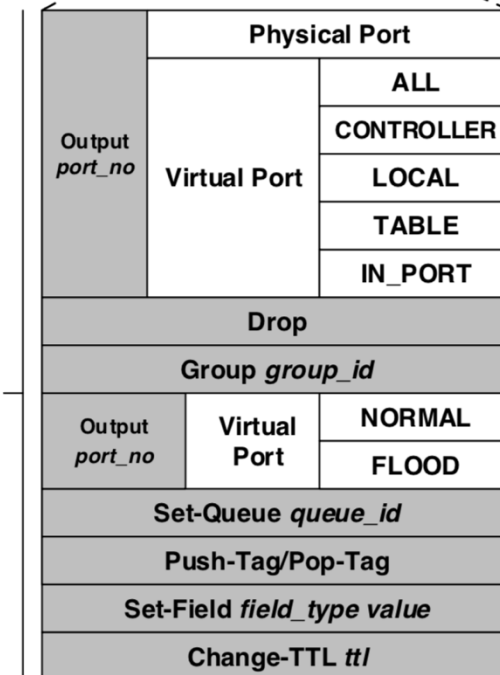
OpenFlow - a forwarding abstraction



- A flow rule is
- ❖ a Match-Action pair
 - ❖ any packet Matched have the associated Action applied to it

Mandatory Actions

Optional Actions



Disaggregation - Benefits

- ❑ New market landscape and value shift
 - ❖ opening a vertically integrated, closed, and proprietary market via open interfaces
 - ❖ shift control from vendors to operators that build networks to meet their users' needs
- ❑ Opportunities for fast innovation
 - ❖ independent evolution and development
 - ❖ the software control of the network can evolve independently of the hardware
 - ❖ lots of new exciting use cases

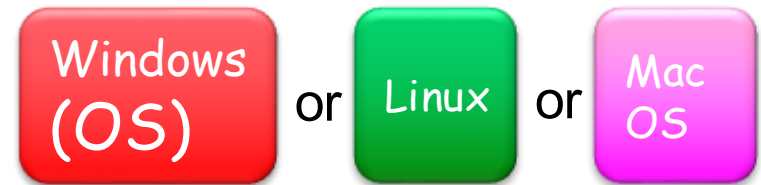


Vertically integrated
Closed, proprietary
Slow innovation
Small industry

Computers



— Open Interface —



— Open Interface —

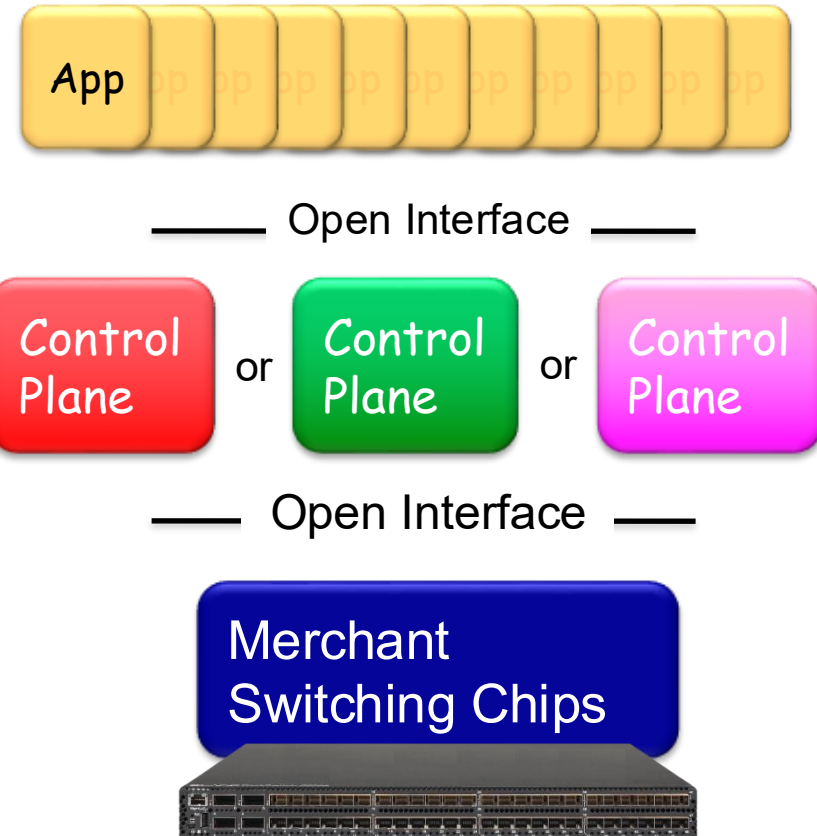
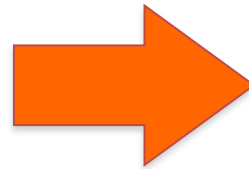


Horizontal
Open interfaces
Rapid innovation
Huge industry

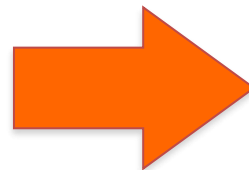
Routers



Vertically integrated
Closed, proprietary
Slow innovation



Horizontal
Open interfaces
Rapid innovation



Control vs Configuration

□ Control

- ❖ making real-time decisions about how to respond to link and switch failures, etc.
- ❖ control plane needs to learn about failure and provide a remedy within milliseconds.

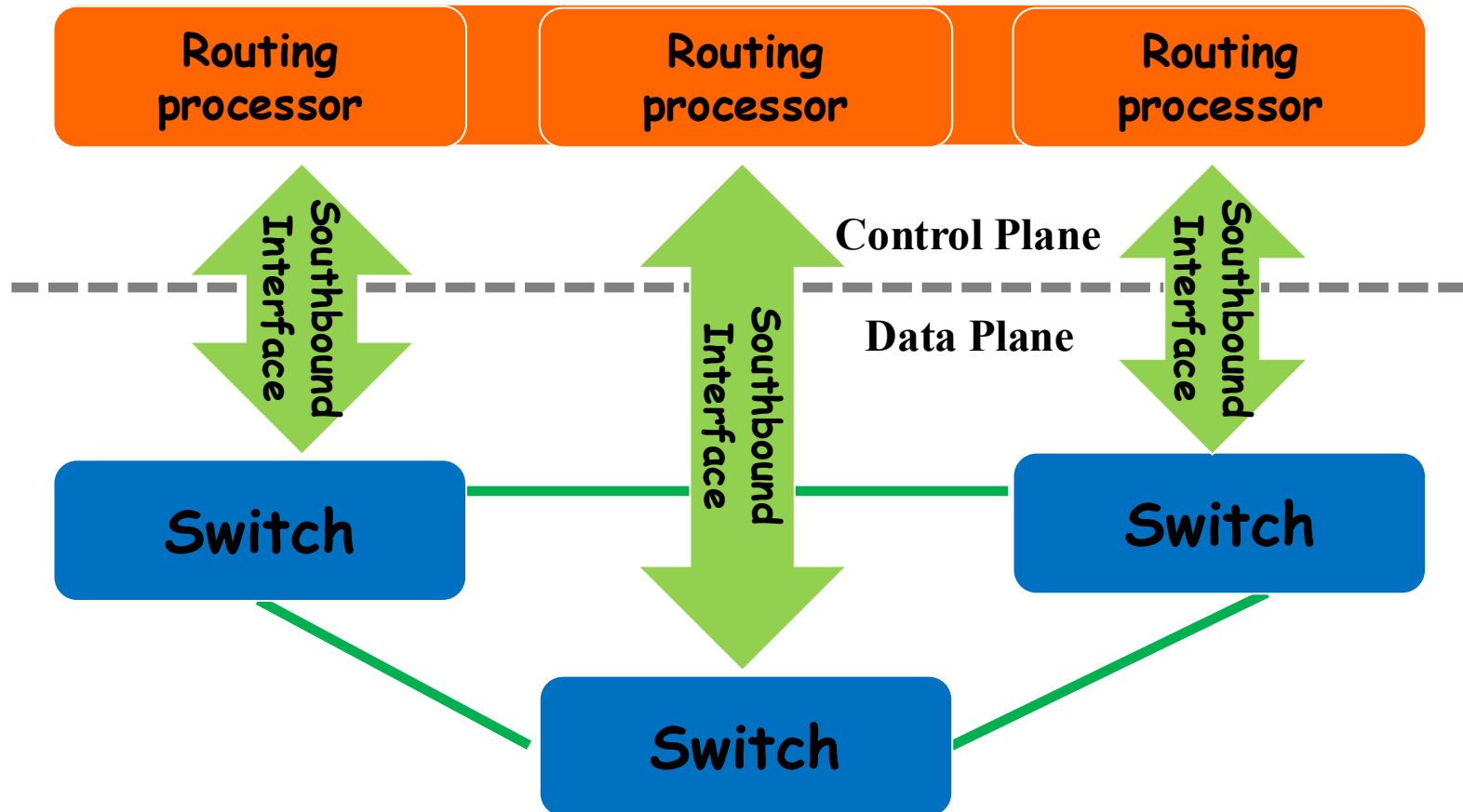
□ Configuration

- ❖ operators need to configure their switches and routers, typically done using CLI to update RIB
- ❖ interface is capable of installing new routes, which on the surface seems to be equivalent to installing a new flow rule.

Where to implement the control plane?

- ❑ Option 1: run the software that implements the control plane *on-switch*
 - ❖ implies switches operate autonomously, and communicate with peer switches throughout the network to construct local routing tables
- ❑ Option 2: make the control plane physically decoupled from the data plane
 - ❖ implies the control plane is implemented *off-switch*, e.g., running the controller in the cloud
 - ❖ possible to make it logically centralized

Principle 2: Centralized Control



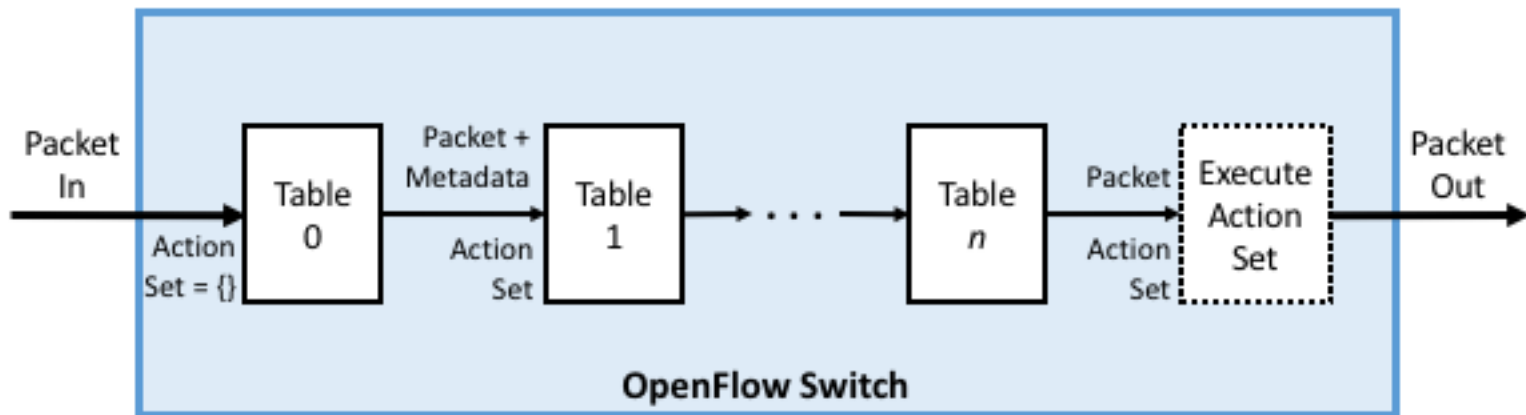
Benefits of Centralization

- Centralized decisions are easier to make
 - ❖ e.g., OSPF (RFC 2328) 244 pages
 - ❖ distributed system part (builds consistent network 100 pages)
 - ❖ routing algorithm (Dijkstra's algorithm 4 pages)
- Logically vs. physically centralized
 - ❖ issues with of a physically centralized controller?
 - ❖ how to implement a logically centralized one?

How to implement the data plane?

❑ From FIB to Forwarding pipeline:

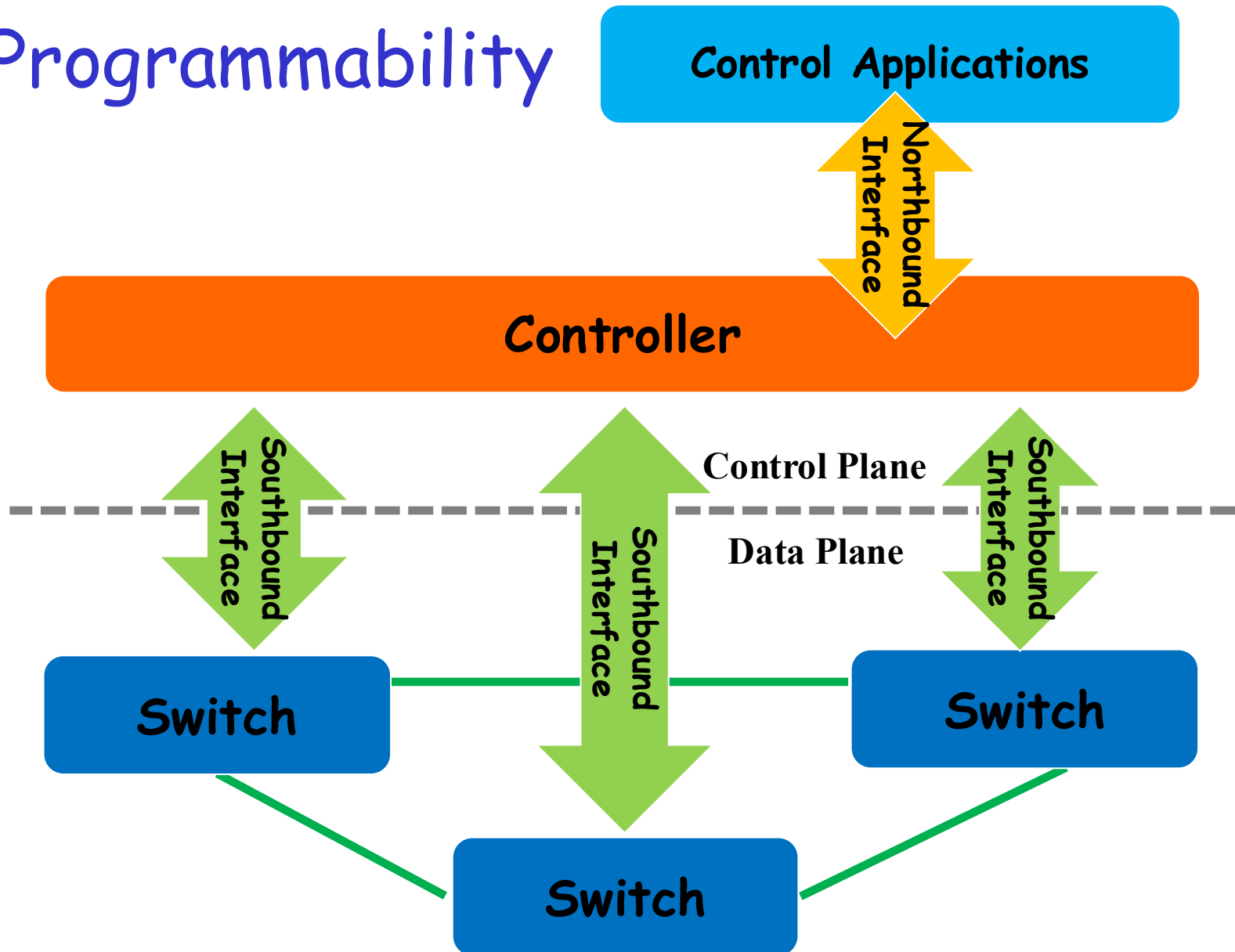
- ❖ each table focuses on a subset of the header fields that might be involved in a given flow rule
- ❖ a packet is processed by multiple tables in sequence to determine how it is forwarded.



How to implement the data plane?

- ❑ Option 1: fixed-function data plane
 - ❖ as header fields are both well-known and at easy-to-compute offsets in every packet
 - ❖ initial idea was purposely data plane agnostic—SDN was entirely focused on opening the control plane to *programmability*
- ❑ Option 2: programmable data plane
 - ❖ performance optimization
 - ❖ potential changes of protocols

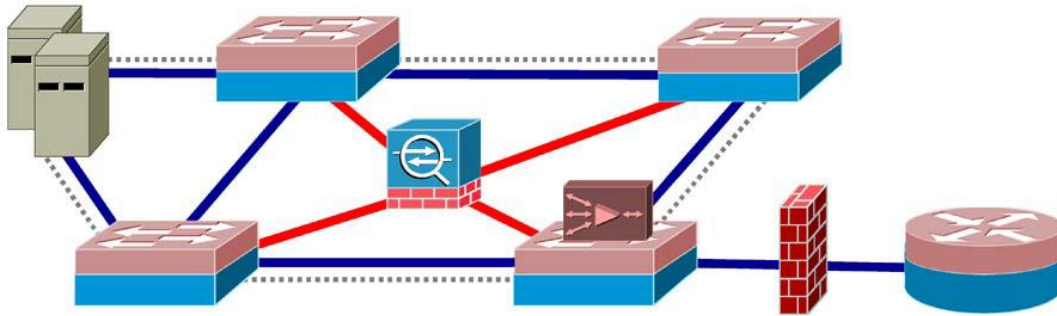
Programmability



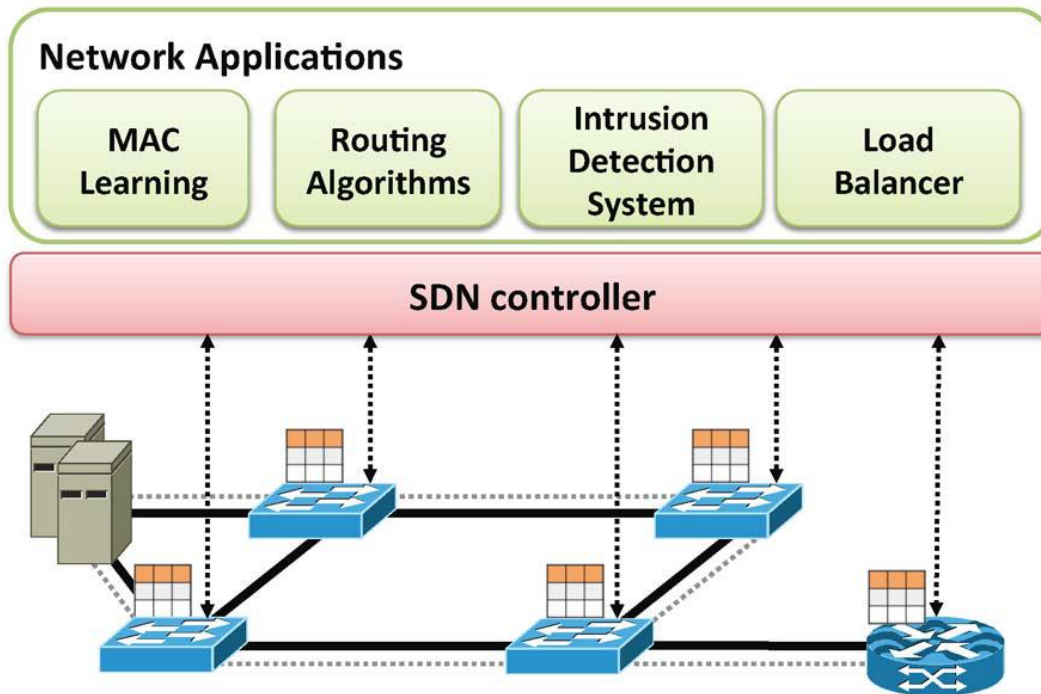
Benefits of Programmability

- ❑ Easy network management
 - ❖ management goals are expressed as policies
 - ❖ debug/check behavior more easily
- ❑ Rapid innovation and fast evolution
 - ❖ enable new services and better performance
 - ❖ detailed configuration are done by controller
- ❑ Control shift: vendor → operators → users
 - ❖ *"Networks will [hopefully] be programmed by many, and operated by few." -- Nick McKeown*

Conventional vs. SDN Networks



Software-Defined Networking



1. Easy to program
2. Easy to manage
3. Cheap to buy

D. Kreutz et al. "Software-Defined Networking: A Comprehensive Survey",
Proceedings of the IEEE, Vol. 103, No. 1, January 2015.

A Quick Summary

- ❑ Principles of Software Defined Networking
 1. Separation of Control Plane and Data Plane
 2. Logically Centralized Control
 3. Programmability
- ❑ The network control problem can be modularized based on these principles into
 - ❖ Specification abstraction
 - ❖ Distribution abstraction
 - ❖ Forwarding abstraction

Three layers of abstractions

□ Specification abstraction

- ❖ allow a control app to express the desired network behavior without implementing it

□ Distribution abstraction

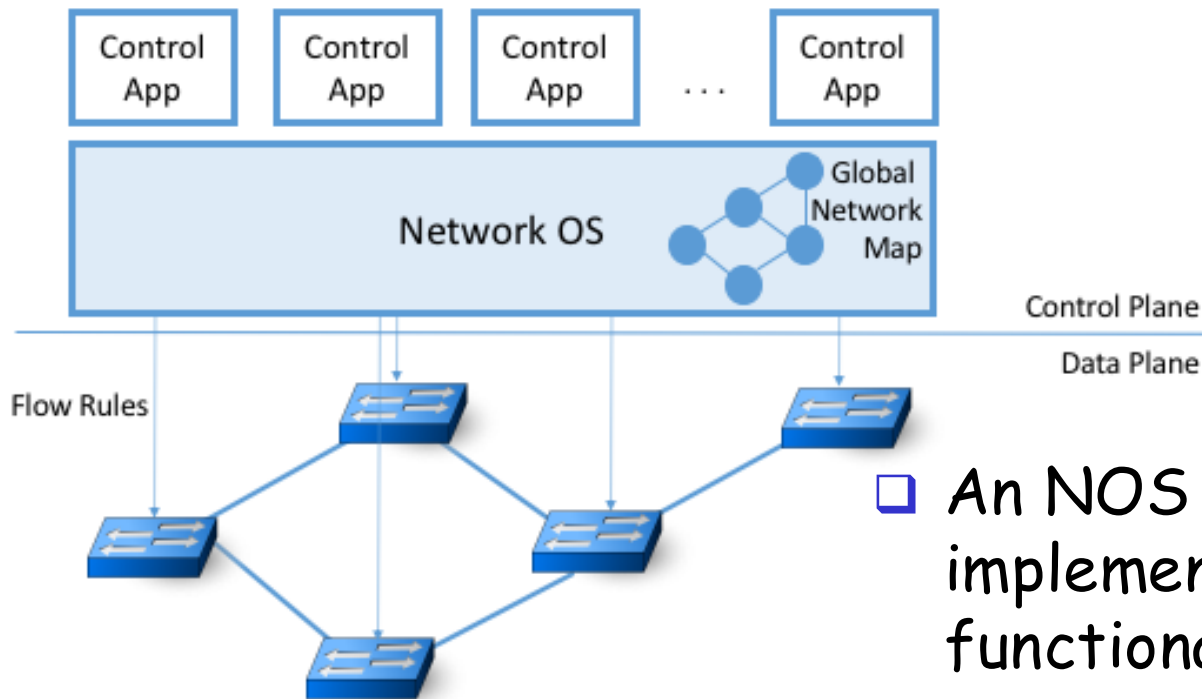
- ❖ shield SDN apps from the distributed states, making distributed control logically centralized

□ Forwarding abstraction

- ❖ allow any forwarding behavior desired by apps while hiding details of the underlying hardware

Network Operating System (NOS)

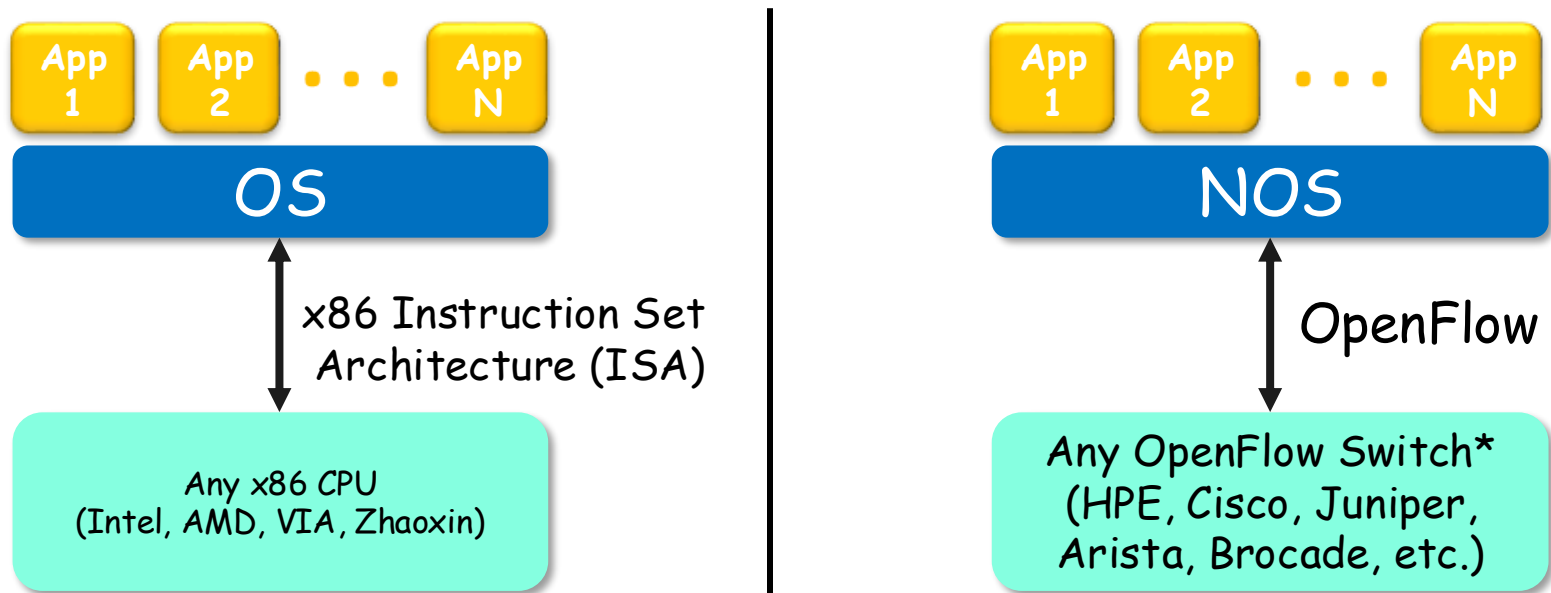
- An OS provides high-level abstractions that make it easier to write applications



- An NOS makes it easier to implement network control functionality: *Control Apps*

SDN: an abstraction for Networking

- Highly simplified analogy with a CPU-centric OS



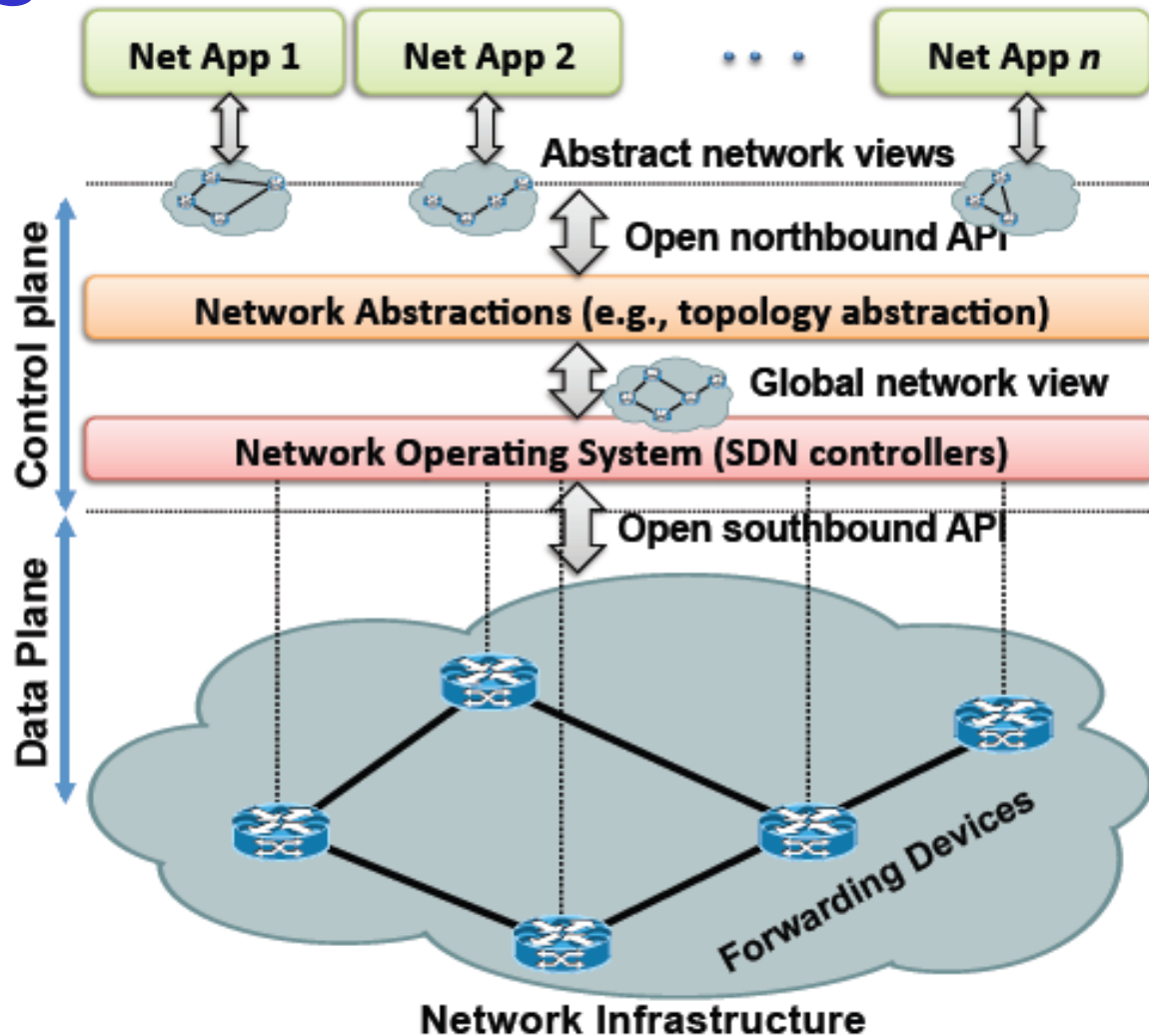
* Network Switch = Packet Processing Unit

Abstractions

□ Specification →

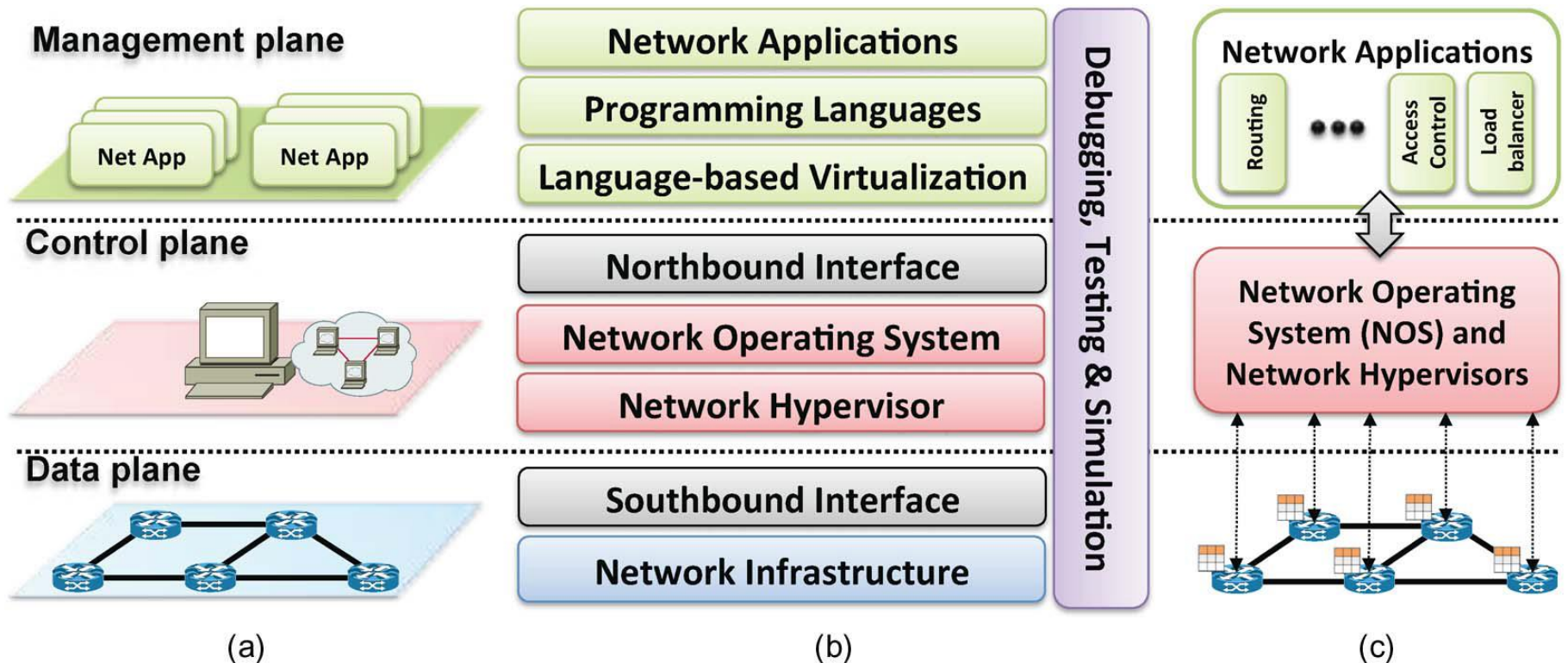
□ Distribution →

□ Forwarding →



D. Kreutz et al. "Software-Defined Networking: A Comprehensive Survey",
Proceedings of the IEEE, Vol. 103, No. 1, January 2015.

Layers in an SDN



D. Kreutz et al. "Software-Defined Networking: A Comprehensive Survey",
Proceedings of the IEEE, Vol. 103, No. 1, January 2015.